

Inigo Quilez :: articles :: genetic algorithms - 2010, 2023



I remember toying around fitting arbitrary primitives to images around 2006 after reading about them in some blog about genetic algorithms. My initial interest was for image compression in the context of the Demoscene, that is, I wanted to compress an image in under 2 kilobytes of code so I could use it as part of some size-limited demo contest.

The idea was to give myself a budget of 500 colored disks and see how they could be best arranged on a pixel canvas to reproduce a photograph. Let me explain: each disk had a position (x, y), a size (s) and a color (r,g,b). That is, six parameters per disk. The disks were blended into the canvas with a gaussian falloff like those used in my previous [3D Gaussian Splatting](#) experiments, but in 2D. You can think of these disks as little smoke particles or sprites or splats, or Photoshop brushes. I'll use "splat" to stay current (edited from "brush" in the first version of this article).

Since my photograph was 256x256 pixels, all brush/gaussian splat parameters did fit in one byte each, for a total of 3 kilobytes (500 splats x 6 parameters/splat x 1 byte/parameter). This was before compression, but hope was I could reduce this a lot by that sorting the points by x and y coordinates in a hilbert, morton or block ordering, and then encoding the position, color and size deltas individually (deinterlaced style), and finally passing it through a regular arithmetic/entropy encoder. I was hoping to make the image fit in 2 kilobytes or so. Eventually this didn't work as well as my [wavelet version](#) a few months later, which I released as a Procedural Executable Graphics demo at the TUM demo party 2007.

Regardless, 3 years later I revisited my experiments and wrote this article with more details. So, let's do this!



Fitting 500 gaussian splats
4000 bytes uncompressed



Reference image

The basic algorithm

Rendering gaussian splats/sprites is rather easy, you can render millions in realtime easily. The key is to find a good way to fit all those splats to a given photograph. In my experiments, I only needed to fit 500 of them, so I didn't need to be very smart about optimizing the process. I started with a "genetic" kind of algorithm (not exactly), which all it did was start with a random set of splats with random parameters, render it to get a mess, and then start refining from there. In my first version of the experiment, at each refinement step I applied a randomized modification to some randomly picked splat. If the resulting image was closer to the reference photograph than the previous version was, I would take this change. If not, I'd revert to the previous one. Then I repeated this many times. It was that simple.

To judge whether the fitted image resembled the actual photograph, I used a simple MSE computation (average difference squared of pixels values). Edit: in modern terms this is equivalent to some sort of random descent through an L2 loss function, but back in 2010 in my mind this was just called "stochastic search for a local minima". Here's the pseudo-code:

```
function fit_image( file_name )  
{
```

```
// init
reference_image = load_image( file_name );
data = randomize(); // arra of 500 * 6 parameters
difference = 1e20;

// compute
while( difference > some_threshold )
{
    new_data = modify( data );
    new_image = render_image( new_data );
    new_difference = compute_MSE( new_image, reference_image );
    if( new_difference < difference )
    {
        data = new_data;
        difference = new_difference;
    }
}

// done
save_to_disk( data );
}
```

Here the "data" is the vector of parameters that best fits the reference image at any given time, and it contains the six parameters of each gaussian (x,y,s,r,g,b) sequentially. This was all coded in C except for the rendering of the gaussian splats in `render_image()`, which was done with OpenGL and `ReadPixels()`. At the time without Compute Shaders nor floating point buffers, this wasn't as bad of a strategy as it is today. These days you can implement all of the steps in the GPU of course, making it much more efficient.

Regardless, even with the slow code, I was able to fit to my image:

Fitting 500 **isotropic** gaussian splats

Reference image

The hierarchical random descent

The most important part of this algorithm was of course the `modify()` function. Today gradient descent is very popular (too popular), but back in 2006 what I implemented was as a random mutation strategy. Basically, navigating the loss function's landscape by taking random steps rather than following the gradient. The magnitude of the randomization starts large with an upper bound of 128. This allowed me to quickly explore as much of the landscape as possible. As the rate of improvements started slowing down, I reduced the magnitude of randomization to 64. This sped up the convergence temporarily. When the process slowed down yet again, I reduced the randomization amplitude to 32. This continued until the image converged. You can see this is as some sort of hierarchical refinement or "learning".

Of course, as the process was stochastic/random, it was not guaranteed that the process would find the best minima possible. That is, if you do this at home, it might be worth trying to start the process from scratch a few times with a different random seeds and follow different evolutionary lines, then pick the best of all runs.

Improvement 1 - importance sampling

An interesting variation to this experiment is to modify the MSE loss function to be more "importance" based. For example, you can multiply the square differences of the pixel values in the MSE/L2 loss function with a weight factor of my choice. This weight factor can be largest in the areas of the face we humans pay most attention to - the eyes. You can do this with a gaussian fall-off centered right between the eyes or any other soft mask of your choosing. This makes those areas more sensitive and the borders of the image more tolerant to errors, forcing more and smaller splats to concentrate around those areas. The following is an example from 2010 using this technique using only 316 splats:



Importance sampled 316 gaussians

Code: <https://www.shadertoy.com/view/4df3D8>

Improvement 2 - Level of Detail

Something that can and often happens is that a number of splats end up covered by other splats, and don't contribute to the image, which is a waste of data of course. The problem is that any type of descent (random or gradient) algorithm won't be able to detect and resolve the situation, for small changes to the parameters of those occluded splats don't change the loss function. One solution is to still accept the new mutated parameter vector if a change didn't worsen the current best fit, rather than only if it improved it. This, in theory, should

eventually untangle the situation and bring the occluded splats to areas where they can contribute. This however is slow and still limited, for the most impactful position the occluded splat could go to might be part of a different local minima, and therefore unreachable.

My solution to this was the following: whenever a gaussian splat candidate was chosen for a mutation of its parameters, I'd compare it to some other random splat that would render at a later time. That is, I'd just compare it to another splat down the parameter vector. If the later was larger than the former, I'd swap them, then continue with the random descent as usual.

This reduced the waster splats, and also made for a more natural way to construct the image, where the largest brush strokes (ie, gaussian splats) happened early and covered as much of the canvas as possible, and then the medium detail splats were added, followed by the many more smaller splats capturing the high frequency details.

Improvement 3 - Anisotropic splats

The circular gaussian splats (or "sprites", or "particle" or "brushes") work fine, but we can do much better. Around 2015 many Shadertoy users started extending the gaussian splatting technique to other primitives - triangles, gabor kernels, etc. However, the first one to realize circular splats could actually be the best primitive if tweaked a bit was Lara Lubsch, with this shader: <https://www.shadertoy.com/view/4tV3RW>. She decided to let the gaussian splats be elliptical of any eccentricity, and be in any arbitrary orientation. This means each gaussian splat would have now not one but two sizes (for its major and minor axis) and also an angle. This brings the number of parameters per splat from six to eight, but it turns this extra expressivity produces incredibly better results.

In 2023 gaussian splatting became popular again through academic research in radiance fields, and that reminded me of my old experiments and Lara's shader. So I gave it a go again, using the exact same code as in 2010 and the same reference image, but adding the two extra parameters per gaussian like Lara did 7 years prior. The results are much better indeed, here's a comparison between the old symmetric (isotropic) gaussian splatting, and Lara's (anisotropic) gaussians of arbitrary aspect ratio and orientation:



500 isotropic gaussian splats

3125 bytes uncompressed

Code: <https://www.shadertoy.com/view/MdfGDH>



500 anisotropic gaussian splats

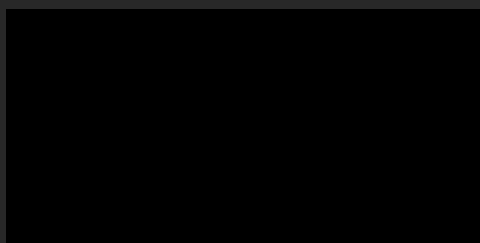
4000 bytes uncompressed

Code: <https://www.shadertoy.com/view/dtSfDD>

To clarify the uncompressed byte sizes of both images, the isotropic gaussians used 9 bits for the x and y coordinates of the splat (each) and 8 bits for splat size and the three color channels (each). The anisotropic gaussians used 9 bits for the x and y coordinates (each), 8 bits for the splat orientation, size of the major axis, size of minor axis and green color channel (each), and 7 bits for the red and blue color channels relative to the green channel (each). You can see the decoding in the Shadertoy links under each image.

Extras

And lastly, this is a timelapse of the fitting process. I stopped recording before reaching full convergence, but it will give you a sense of what the process looks like, in case you implement it yourself. Also, it just looks cool, and that's enough of a reason to share it :)



inigo quilez - learning computer graphics since 1994